CE-1103 · Lección 06

Grafos

Modelar relaciones y recorrerlas con algoritmos



Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

// introducción

¿Qué aprenderemos hoy?

1. Describir un grafo y sus conceptos: ponderado, grado, ruta, ciclo y conexidad
2. Representar un grafo con matriz y con lista de adyacencia
3. Recorrer un grafo con Breadth-First (cola) y Depth-First (pila)
4. Calcular caminos más cortos (Dijkstra, Floyd) y el cierre transitivo (Warshall)
5. Obtener un árbol de expansión mínimo con Prim y Kruskal

// introducción

¿Dónde se usan los grafos?

Siempre que haya **entidades** y **relaciones** entre ellas:

Sociología

Personas y sus relaciones; por ejemplo, identificar redes criminales.

Química

Átomos y enlaces que forman una molécula.

Geografía

Ciudades y carreteras; rutas entre lugares.

Ingeniería eléctrica

Componentes y conexiones de un circuito.

Ingeniería industrial

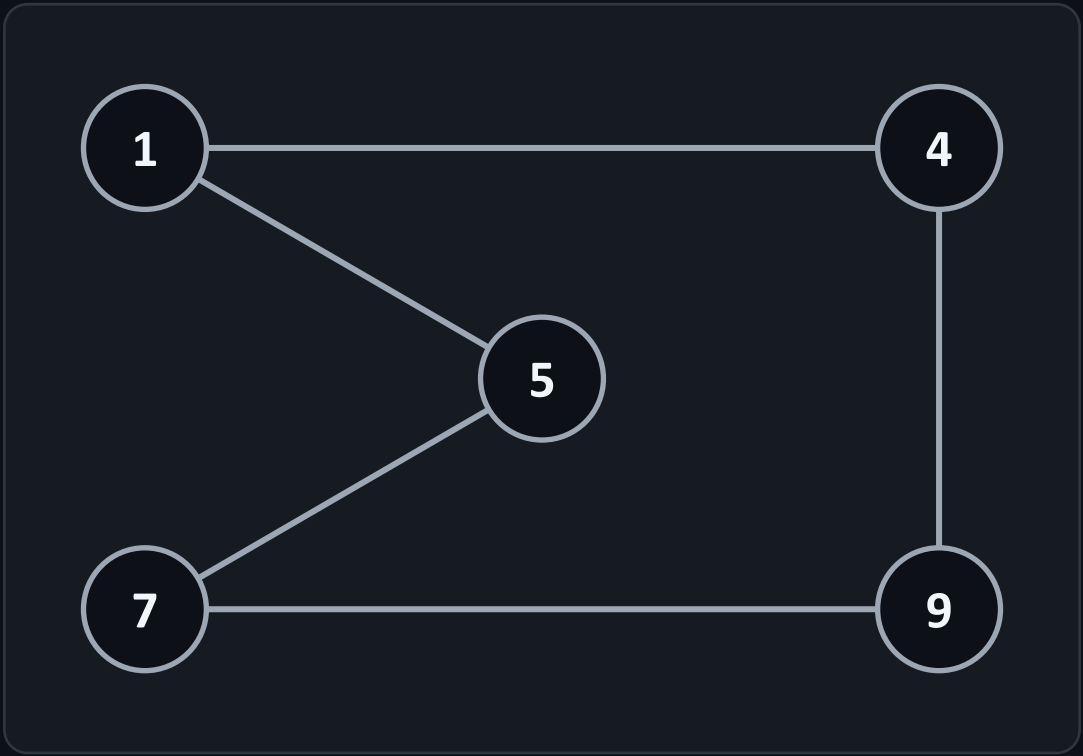
Tareas de un proceso y sus dependencias.

Computación

Páginas web y enlaces; computadoras y cables de red.

// conceptos

¿Qué es un grafo?



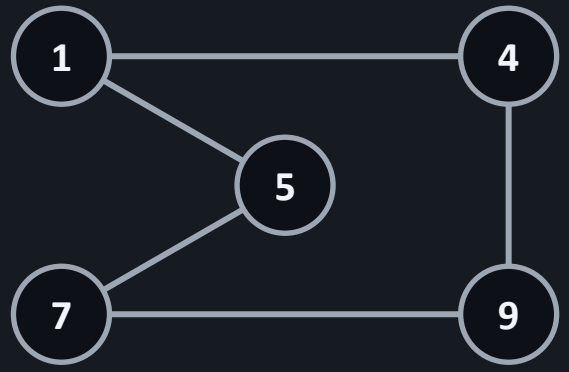
- **V:** conjunto de vértices (o nodos)
- **A:** conjunto de aristas (o bordes)

```
G = { V, A }
V = { 1, 4, 5, 7, 9 }
A = { (1,4), (4,1), (1,5), (5,1),
      (4,9), (9,4), (5,7), (7,5),
      (7,9), (9,7) }
```

Un grafo agrupa **entidades** físicas o conceptuales (vértices) y las **relaciones** entre ellas (aristas).

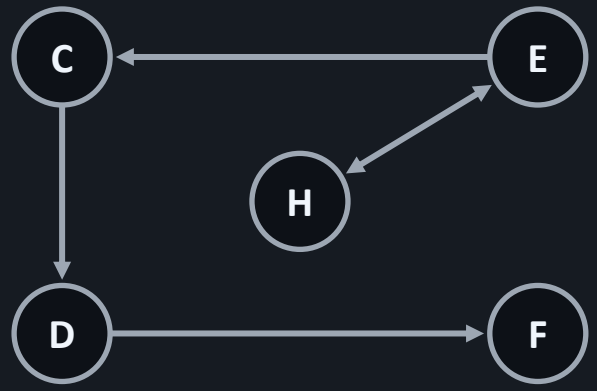
// conceptos

No dirigidos, dirigidos y ponderados



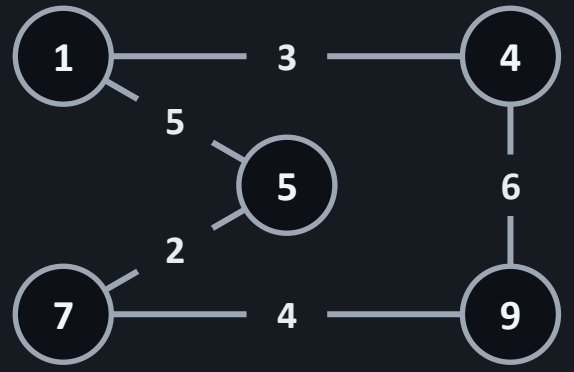
No dirigido

Sin dirección: los nodos conectados son **adyacentes**.



Dirigido

La flecha indica la **dirección** de la relación.

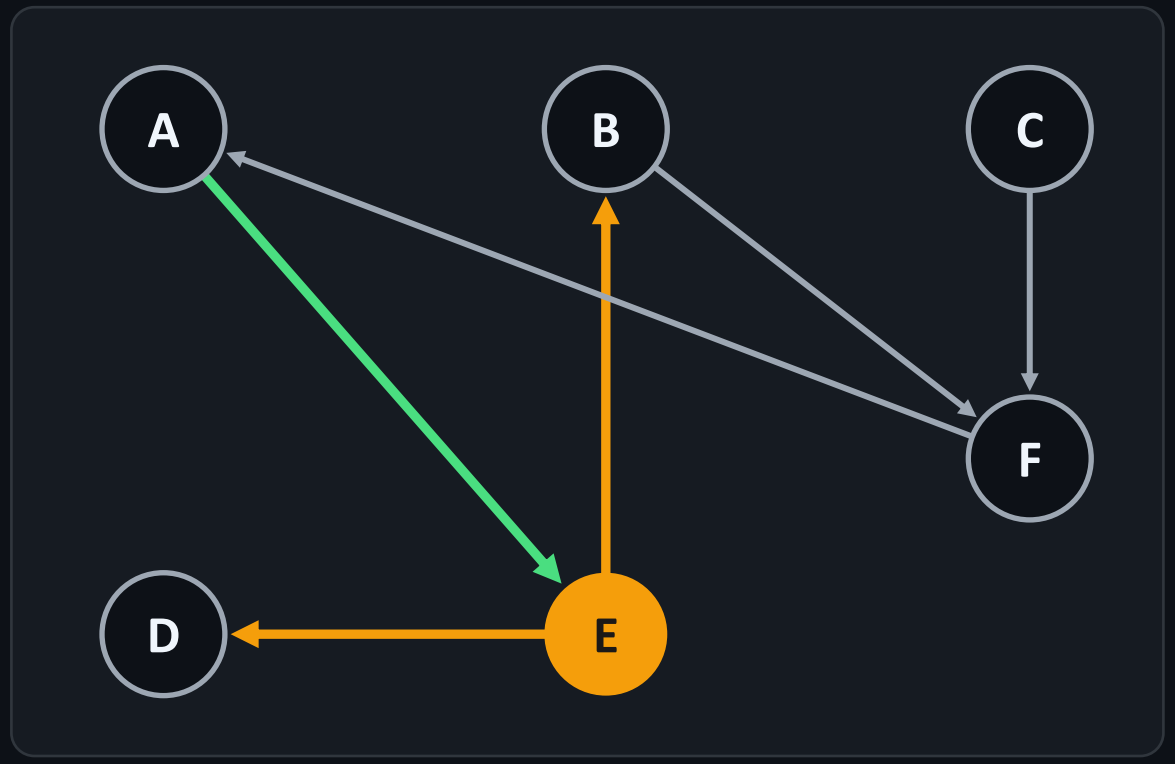


Ponderado

Cada arista tiene un **peso**: la magnitud de la relación.

> En un grafo no dirigido la arista (v, w) es igual que (w, v); en uno dirigido, no. En uno **ponderado**, cada arista lleva un peso.

Grado de un vértice

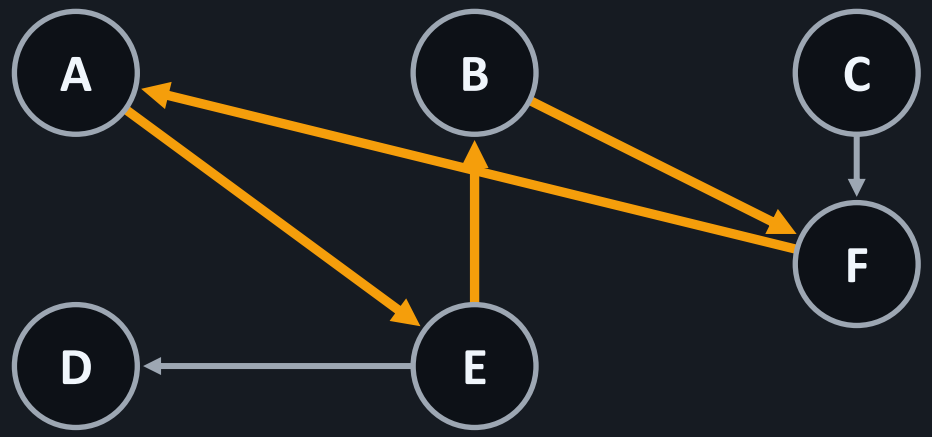


- **No dirigido:** número de aristas que contiene v
- **Dirigido:** entrada (aristas que llegan) y salida (aristas que salen)

Vértice	Entrada	Salida
A	1	1
B	1	1
C	0	1
D	1	0
E	1	2
F	2	1

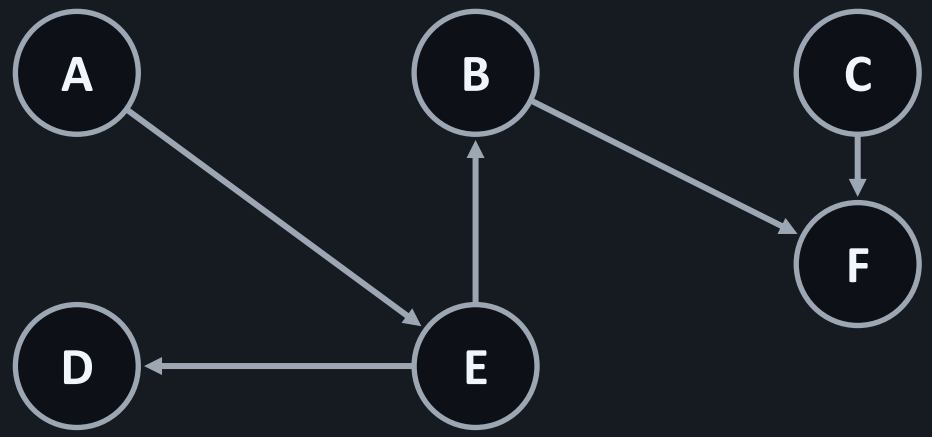
// conceptos

Ruta, ciclo y DAG



Con ciclo

Ruta simple A → E → B → F → A: empieza y termina en el mismo nodo.



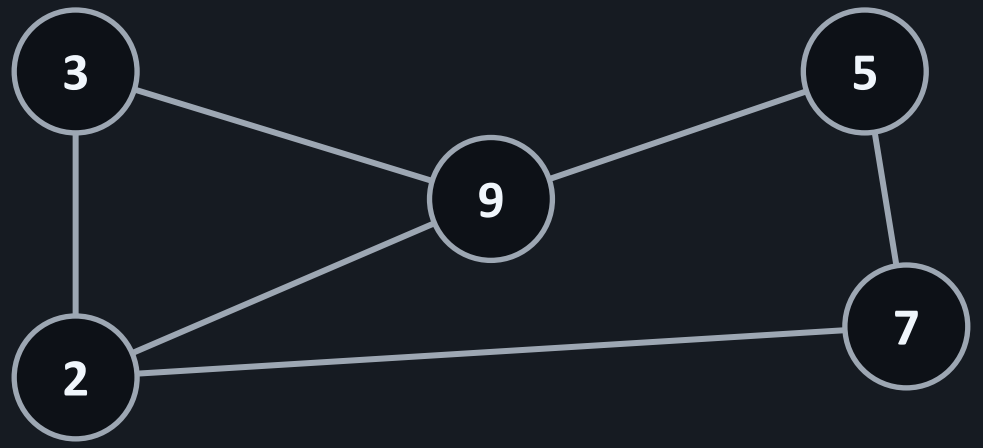
DAG

Grafo dirigido **acíclico**: no hay forma de volver a un nodo siguiendo las flechas.

Ruta: P = (v0, v1, …, vn). Es **simple** si sus vértices son distintos.

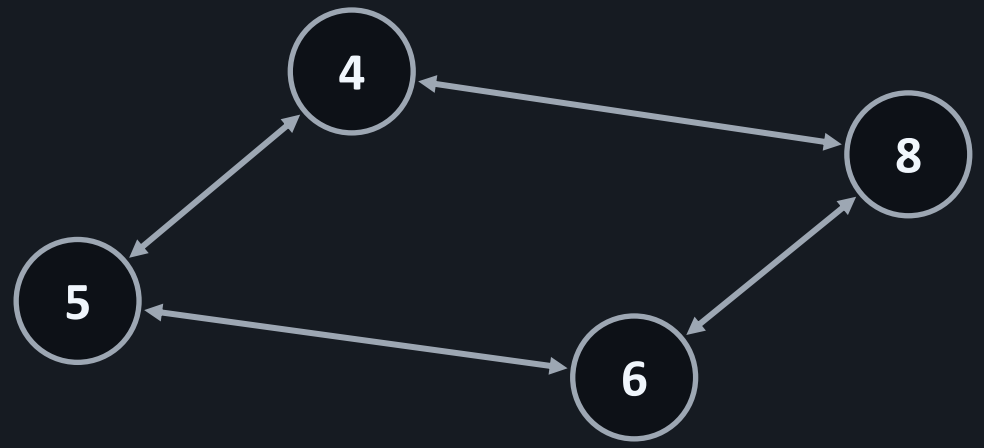
// conceptos

Grafos conexos y fuertemente conexos



Conexo

Existe un camino entre cualquier par de nodos.



Fuertemente conexo

Dirigido y conexo: de cualquier nodo se llega a cualquier otro siguiendo las flechas.

Comprobar si un grafo es **conexo** es una tarea común: ¿hay camino entre cualquier par de vértices?

// representación

Dos formas de representar un grafo

Matriz de adyacencia

- Matriz bidimensional de n × n
- `a[i][j] = 1` si hay arco de vi a vj
- Conviene en grafos **densos** (muchas aristas)

vs

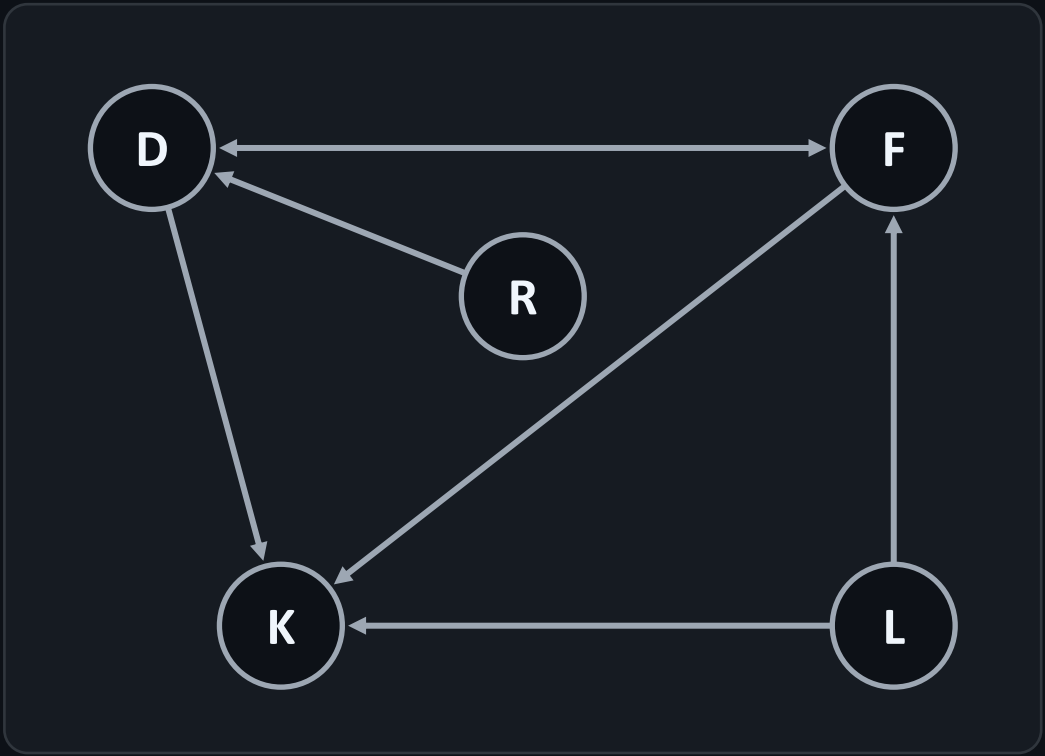
Lista de adyacencia

- Representación dinámica: lista vinculada
- Cada elemento es un nodo con la lista de sus relaciones
- Conviene en grafos **dispersos** (pocas aristas)

> Elegir una u otra depende del tipo de grafo y de las operaciones que se realizarán.

// representación

Matriz de adyacencia



Matriz A (fila = origen, columna = destino)

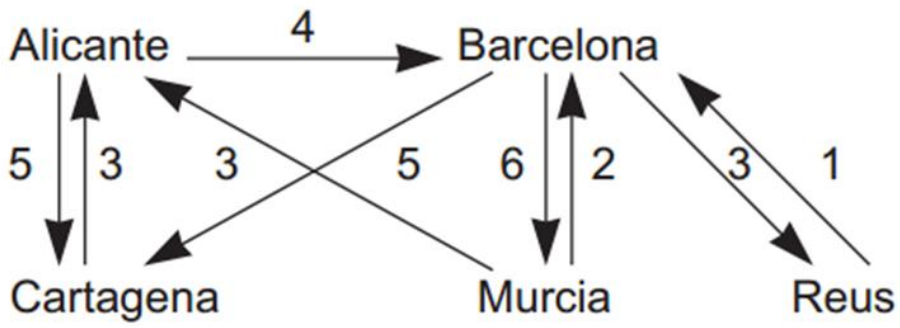
	D	F	K	L	R
D	0	1	1	0	0
F	1	0	1	0	0
K	0	0	0	0	0
L	0	1	1	0	0
R	1	0	0	0	0

`a[i][j] = 1` si existe un arco de vi a vj; `a[i][j] = 0` si no existe.

// representación

Matriz de un grafo ponderado

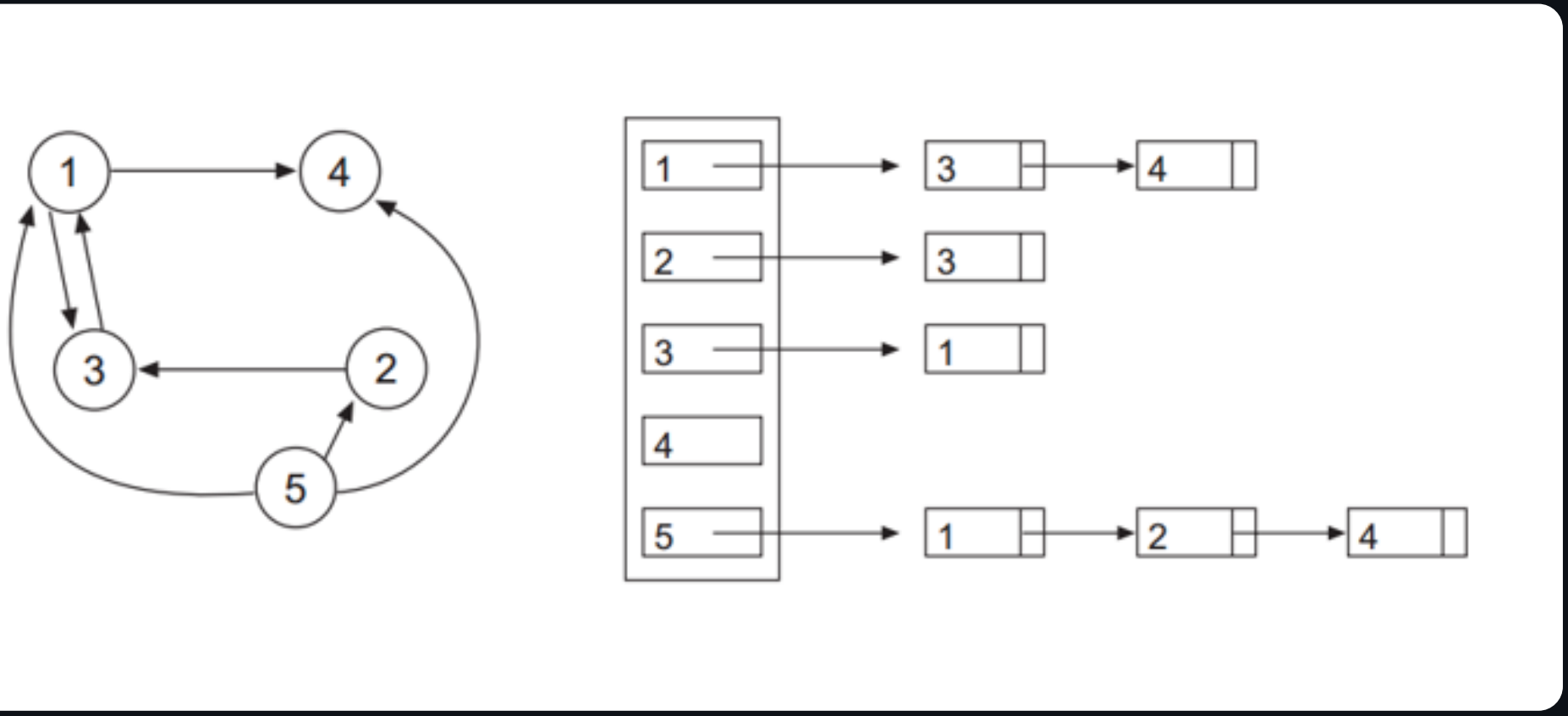
$$P = \begin{vmatrix} 0 & 4 & 5 & 0 & 0 \\ 0 & 0 & 3 & 6 & 3 \\ 3 & 0 & 0 & 0 & 0 \\ 5 & 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{vmatrix}$$



Si el grafo es ponderado, cada celda guarda el peso del arco (0 = no hay arco)

// representación

Lista de adyacencia



Cada nodo guarda la lista de los nodos con los que se relaciona (él es el origen)

Recorridos: Breadth-First y Depth-First

Breadth-First (BFS)

- Usa una **cola** (FIFO)
- Marque y encole el nodo de inicio v
- Saque w, procéselo y encole sus adyacentes sin marcar
- Primero los adyacentes de v, luego los de sus adyacentes…

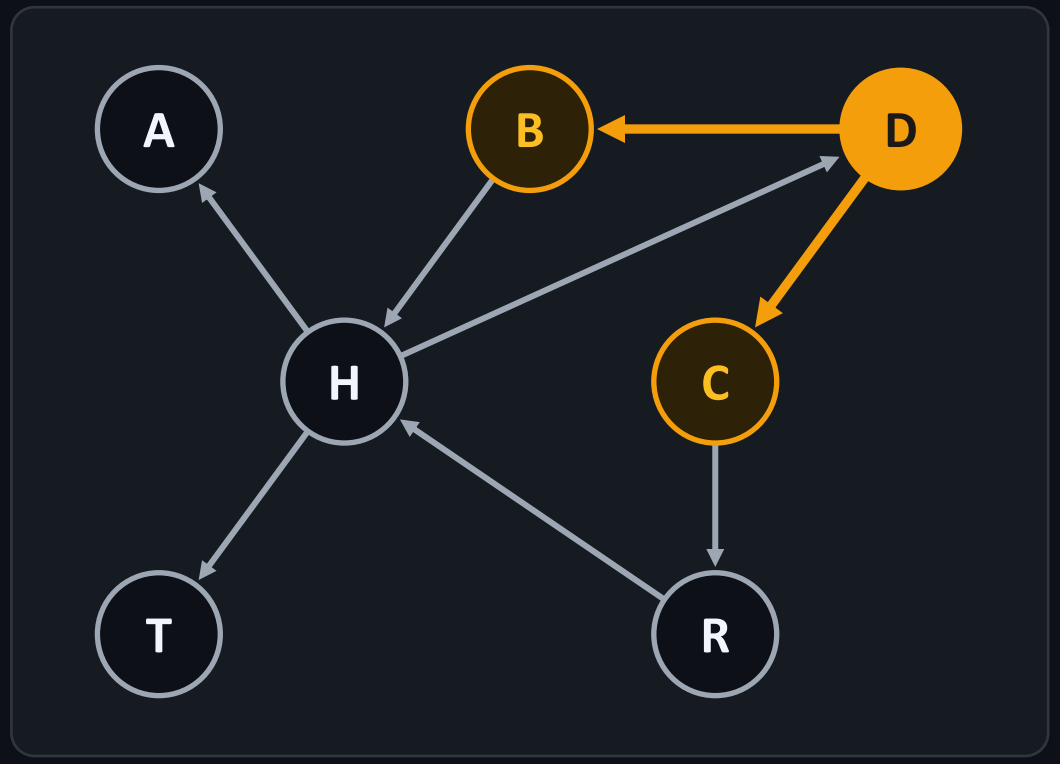
vs

Depth-First (DFS)

- Usa una **pila** (LIFO)
- Marque v como visitado y empújelo a la pila
- Saque el tope y empuje sus adyacentes no visitados
- Continúa hasta que la pila esté vacía

> Recorrer = visitar todos los nodos accesibles desde un nodo de inicio; termina cuando no quedan nodos por visitar.

BFS desde D: se procesa D



Visitado | Procesando | En cola

Cola (FIFO)



frente

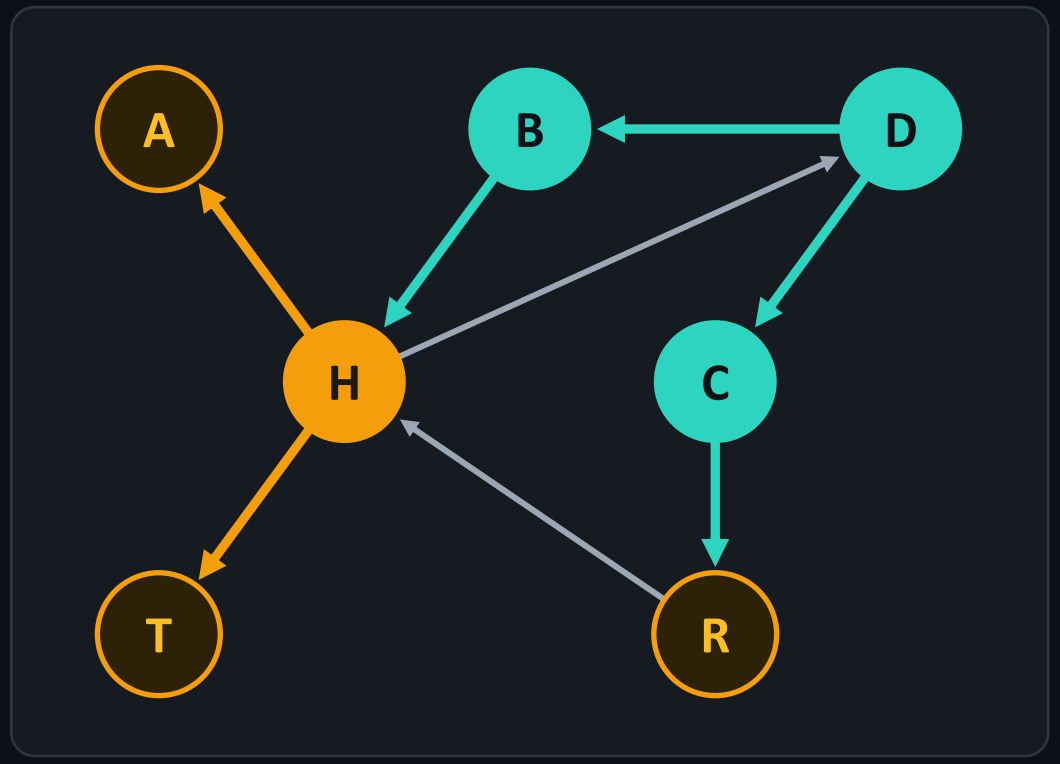
Orden de visita



> Se saca D de la cola, se procesa y se encolan sus adyacentes sin marcar: **B** y **C**.

> **Inicio:** se marca D y se pone en cola. Los vecinos de D se procesan antes que los de B o C.

BFS desde D: se procesa H



Visitado | Procesando | En cola

Cola (FIFO)



frente

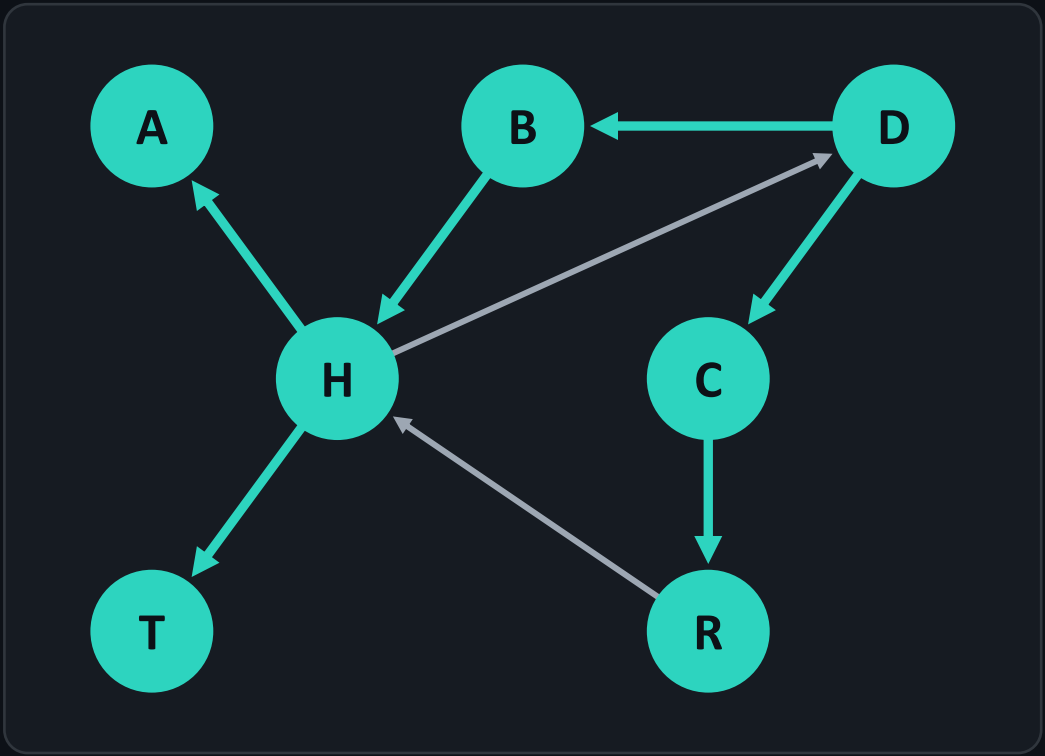
Orden de visita



H tiene tres adyacentes: **D** (ya marcado), **A** y **T**, que se encolan. R ya estaba en la cola.

Los nodos salen de la cola en el mismo orden en que entraron: R, A, T.

BFS desde D: recorrido completo



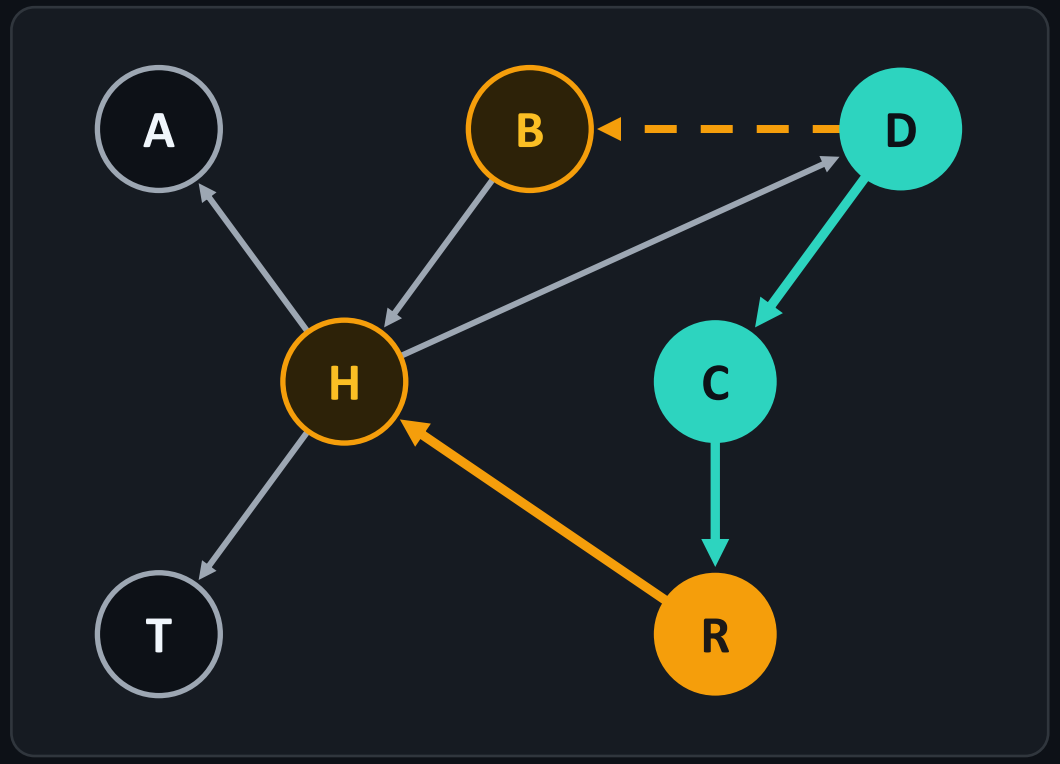
Paso	Procesa	Cola después
0	—	D
1	D	B C
2	B	C H
3	C	H R
4	H	R A T
5	R	A T
6	A	T
7	T	vacía

Visitado

Arista que descubrió el nodo

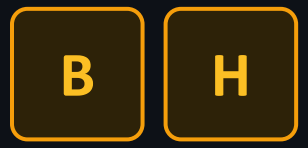
Orden de visita: D, B, C, H, R, A, T. Cada nodo se visita una sola vez.

DFS desde D: se profundiza antes de volver



Visitado | Procesando | En pila

Pila (LIFO)



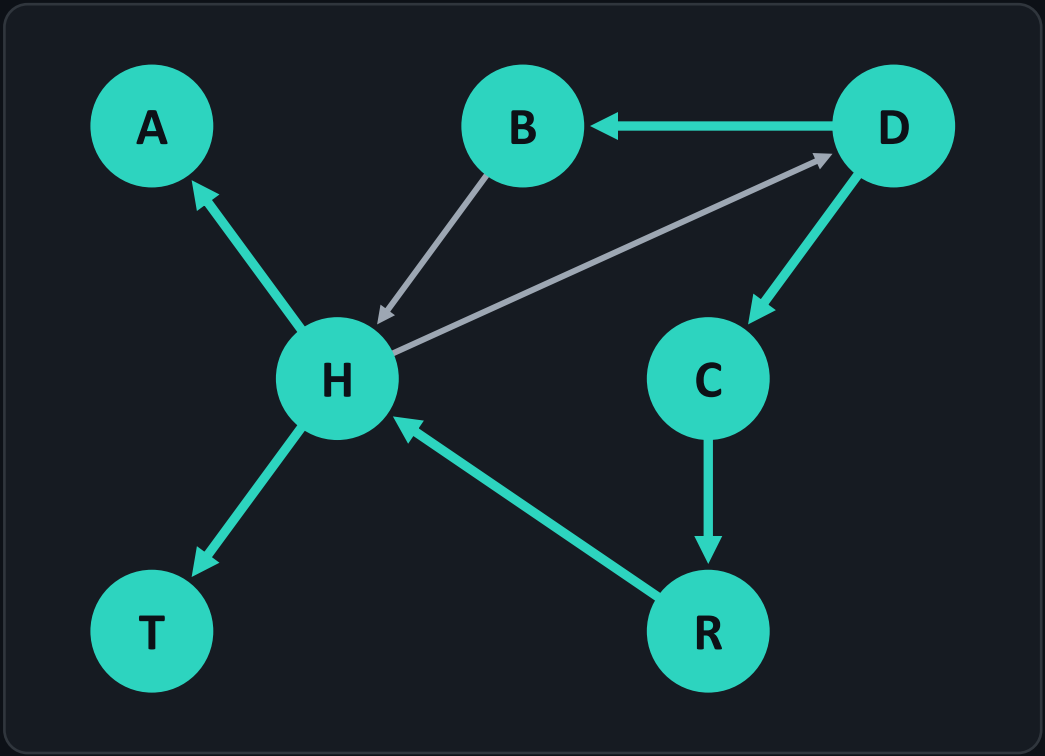
Orden de visita



> Se procesa R y se empuja H. **B** quedó en el fondo de la pila: se visitará al final.

LIFO: el último nodo empujado se visita primero, así que el recorrido avanza en profundidad (D → C → R → H).

DFS desde D: recorrido completo



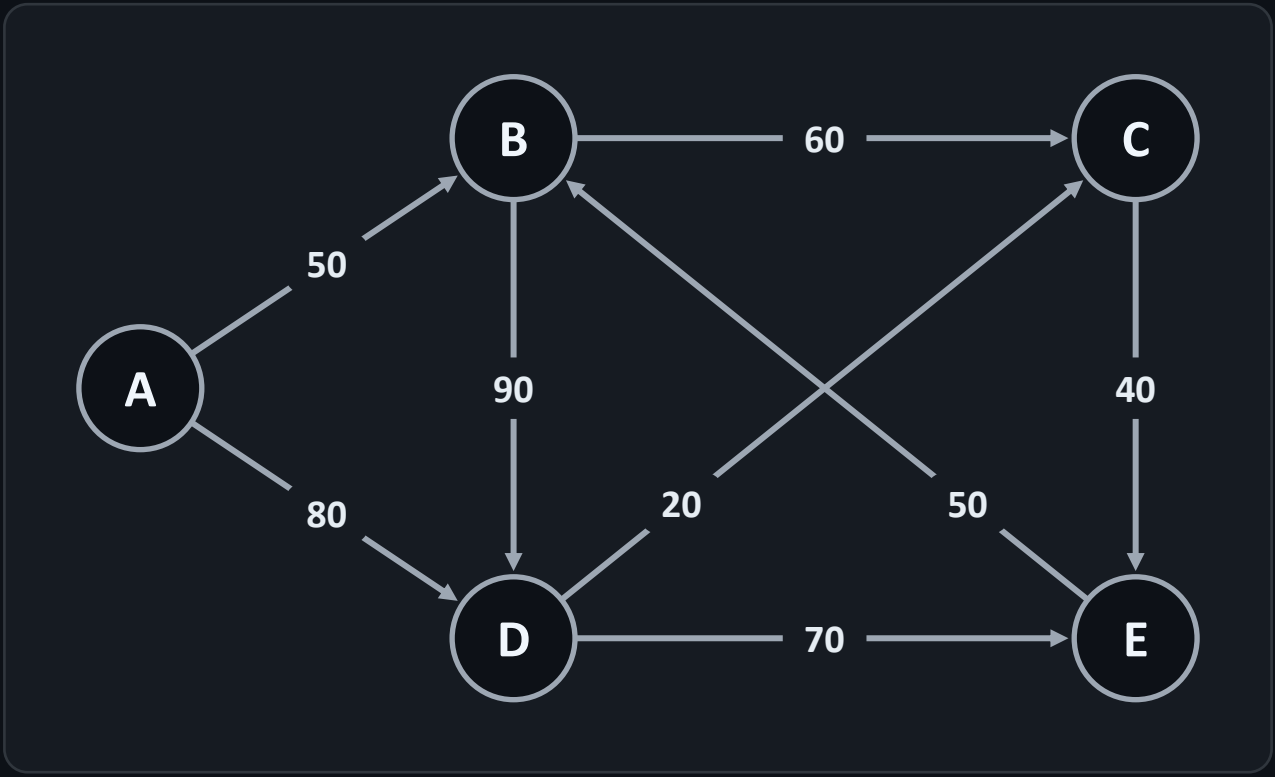
Paso	Visita	Pila después
0	—	D
1	D	B C
2	C	B R
3	R	B H
4	H	B A T
5	T	B A
6	A	B
7	B	vacía

Visitado

Arista por la que se empujó el nodo

Orden de visita: D, C, R, H, T, A, B. B, apilado al inicio, se visita al final.

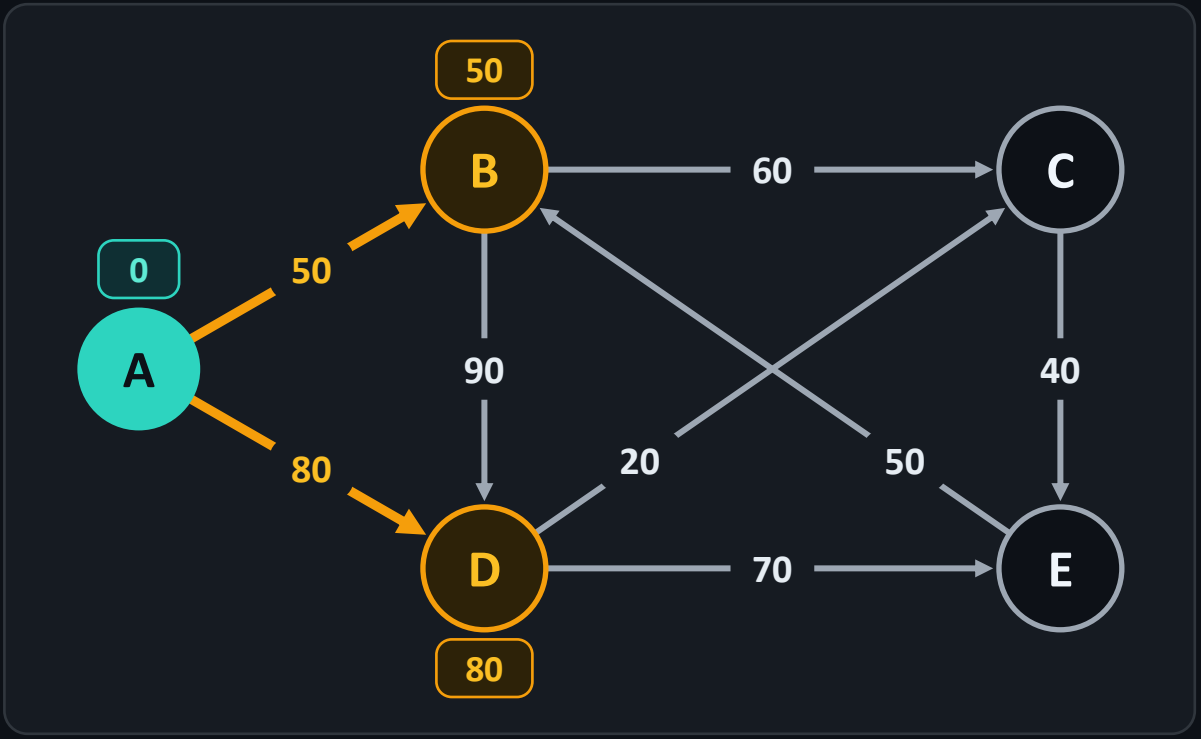
Camino más corto: Dijkstra



- Grafo **dirigido y ponderado**, con pesos positivos
- Longitud del camino = **suma de los pesos** de sus aristas
- Dijkstra: camino más corto de un origen a **todos** los demás nodos
- Edsger Dijkstra (1930-2002), informático holandés

Ejemplo: ir de **A** a **E** con el menor costo total.

Dijkstra: se fija el origen A



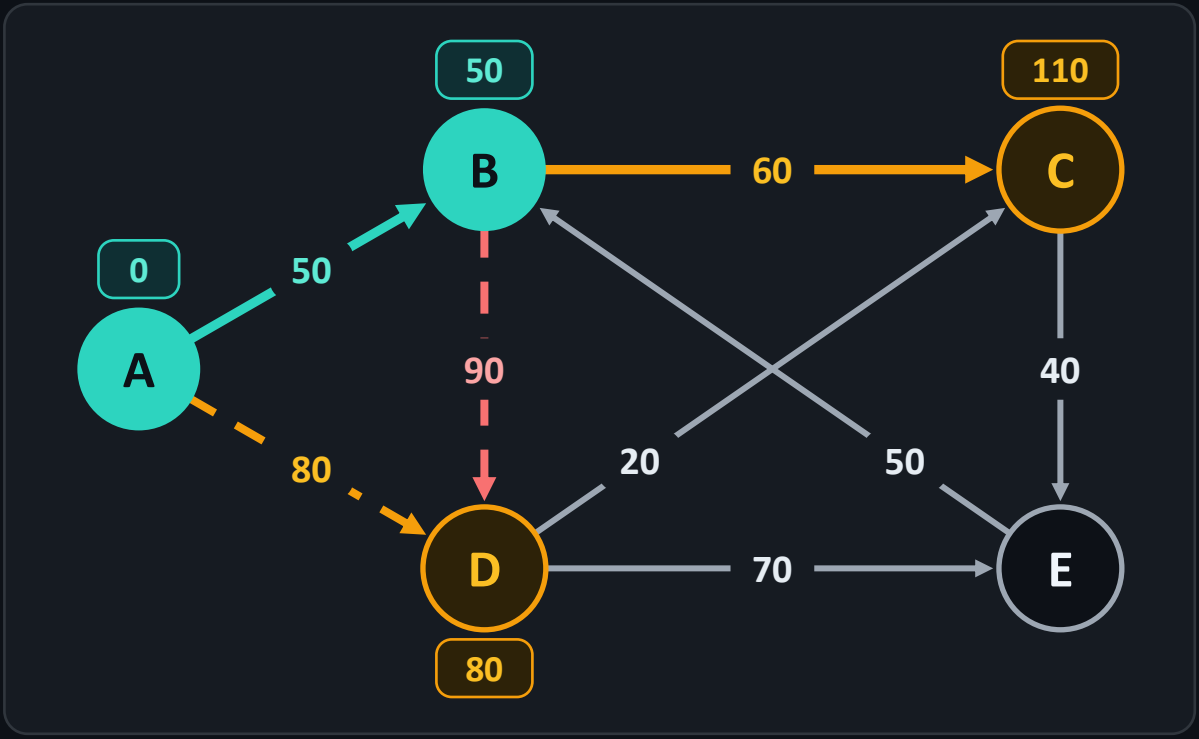
Fijado (final) Temporal

Vértice	Temporal	Final	Previo
A	0	0	—
B	50	—	A
C	∞	—	—
D	80	—	A
E	∞	—	—

Se parte de A (0). Sus adyacentes reciben su peso: **B = 50** y **D = 80**.

Algoritmo voraz (greedy): en cada paso se fija el nodo pendiente con el **menor** valor temporal.

Dijkstra: se fija B (50)



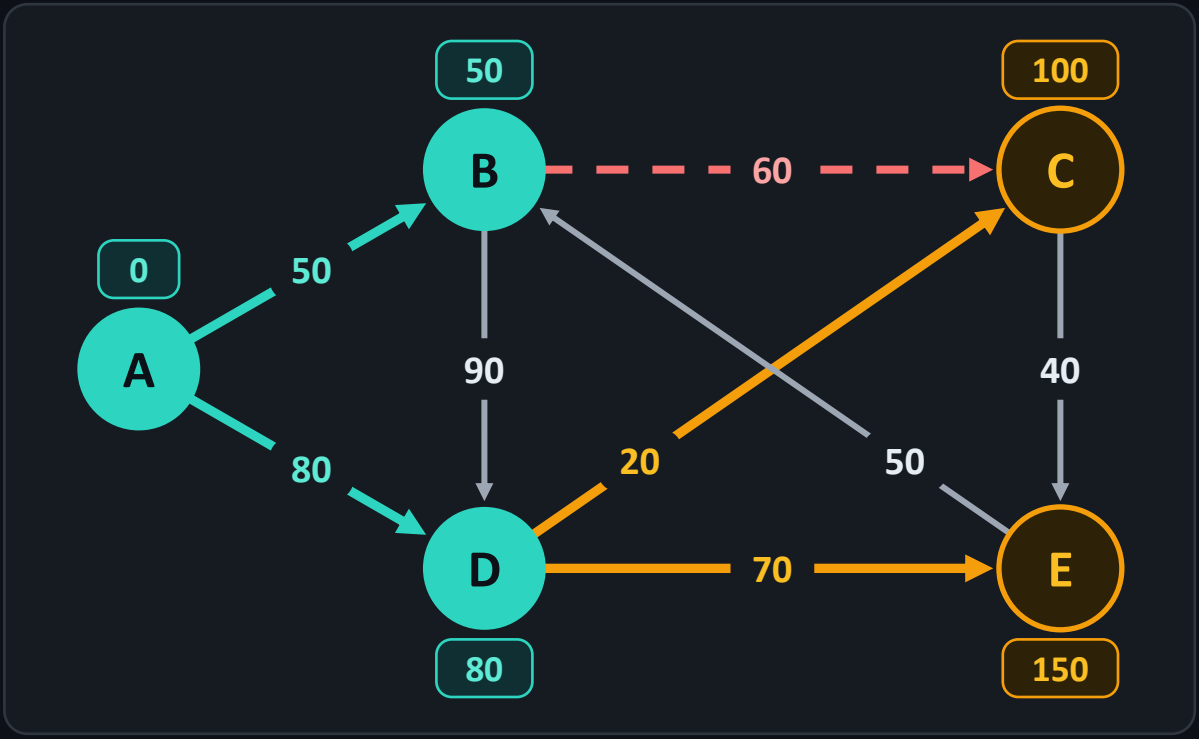
Fijado (final) | Temporal | Descartado

Vértice	Temporal	Final	Previo
A	0	0	—
B	50	50	A
C	110	—	B
D	80	—	A
E	∞	—	—

Se fija B (50). **C = 50 + 60 = 110**; D no mejora (50 + 90 = 140 > 80).

Un valor temporal solo se reemplaza si el nuevo camino es **más corto**.

Dijkstra: se fija D (80)



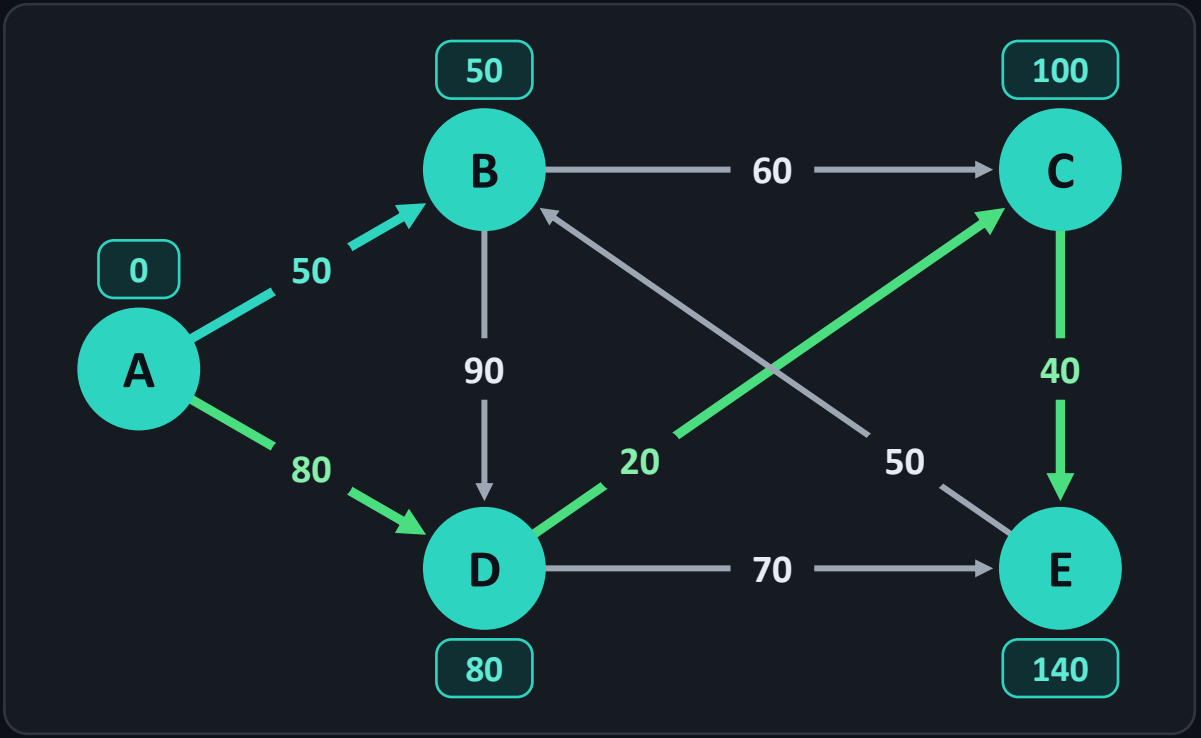
Fijado (final) | Temporal | Descartado

Vértice	Temporal	Final	Previo
A	0	0	—
B	50	50	A
C	100	—	D
D	80	80	A
E	150	—	D

Se fija D (80). **C = 80 + 20 = 100** mejora los 110; E = 80 + 70 = 150.

Mejora: A → D → C cuesta 100, menos que A → B → C (110).

Dijkstra: se fijan C (100) y E (140)



Fijado (final) Camino A → E

Vértice	Temporal	Final	Previo
A	0	0	—
B	50	50	A
C	100	100	D
D	80	80	A
E	140	140	C

Se fija C (100): **E = 100 + 40 = 140** mejora los 150. Luego se fija E.

Camino más corto de A a E: A → D → C → E, con costo **140**.

// camino más corto: dijkstra

Dijkstra: estructura general

```
Foreach node set distance[node] = HIGH
SettledNodes = empty
UnSettledNodes = empty

Add sourceNode to UnSettledNodes
distance[sourceNode] = 0
while (UnSettledNodes is not empty) {
    evaluationNode = getNodeWithLowestDistance(UnSettledNodes)
    remove evaluationNode from UnSettledNodes
    add evaluationNode to SettledNodes
    evaluatedNeighbors(evaluationNode)
}
```

Inicio

Todas las distancias en HIGH (∞), salvo la del origen: 0.

Conjuntos

SettledNodes: ya fijados.
UnSettledNodes: candidatos.

> Cada vuelta del `while` fija el nodo de **menor distancia** entre los candidatos.

Dijkstra: evaluar los vecinos

```
evaluatedNeighbors(evaluationNode) {
    Foreach destinationNode which can be reached via an edge
            from evaluationNode AND is not in SettledNodes {
        edgeDistance = getDistance(edge(evaluationNode, destinationNode))
        newDistance = distance[evaluationNode] + edgeDistance
        if (distance[destinationNode] > newDistance) {
            distance[destinationNode] = newDistance
            predecessor[destinationNode] = evaluationNode
            add destinationNode to UnSettledNodes
        }
    }
}
```

Si el camino por `evaluationNode` es **más corto**, se actualizan la distancia y el predecesor.

El **predecesor** permite reconstruir el camino al final: E ← C ← D ← A.

// floyd y warshall

Floyd: todos los caminos más cortos

D: distancias mínimas (n × n)

```
D0[i][j] = c_ij   si existe el arco (vi, vj)
         = ∞      si no existe
D0[i][i] = 0
Dk[i][j] = mínimo( Dk-1[i][j],
                   Dk-1[i][k] + Dk-1[k][j] )
```

Q: predecesores (n × n)

```
Q0[i][j] = 0   si i = j o D0[i][j] = ∞
         = i   en los demás casos
Qk[i][j] = Qk-1[i][j]   si Dk-1[i][j] ≤ Dk-1[i][k] + Dk-1[k][j]
         = Qk-1[k][j]   en caso contrario
```

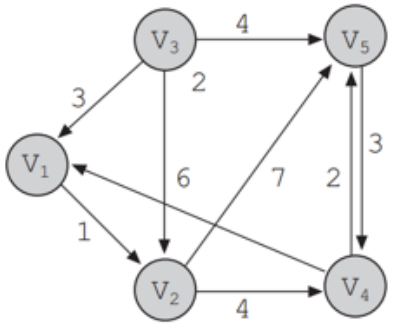
¿Y todos los pares?

¿Dijkstra desde cada nodo? Sirve, pero hay algo más directo: **Floyd**.

Idea

Programación dinámica: en el paso k se prueba si pasar por vk acorta el camino.

Floyd: ejemplo con cinco vértices



$D_k[i, j]$ = minimum($D_{k-1}[i,j]$, $D_{k-1}[i,k] + D_{k-1}[k,j]$)

j

D_0	V_1	V_2	V_3	V_4	V_5
V_1	0	1	∞	∞	∞
V_2	∞	0	∞	4	7
V_3	3	2	0	∞	4
V_4	6	∞	∞	0	2
V_5	∞	∞	∞	3	0

Q_0	V_1	V_2	V_3	V_4	V_5
V_1	0	1	0	0	0
V_2	0	0	0	2	2
V_3	3	3	0	0	3
V_4	4	0	0	0	4
V_5	0	0	0	5	0

i

D_1	V_1	V_2	V_3	V_4	V_5
V_1	0	1	∞	∞	∞
V_2	∞	0	∞	4	7
V_3	3	2	0	∞	4
V_4	6	7	∞	0	2
V_5	∞	∞	∞	3	0

Q_1	V_1	V_2	V_3	V_4	V_5
V_1	0	1	0	0	0
V_2	0	0	0	2	2
V_3	3	3	0	0	3
V_4	4	1	0	0	4
V_5	0	0	0	5	0

D1[4,2] = mín(∞, D0[4,1] + D0[1,2]) = 6 + 1 = 7, y Q1[4,2] = 1

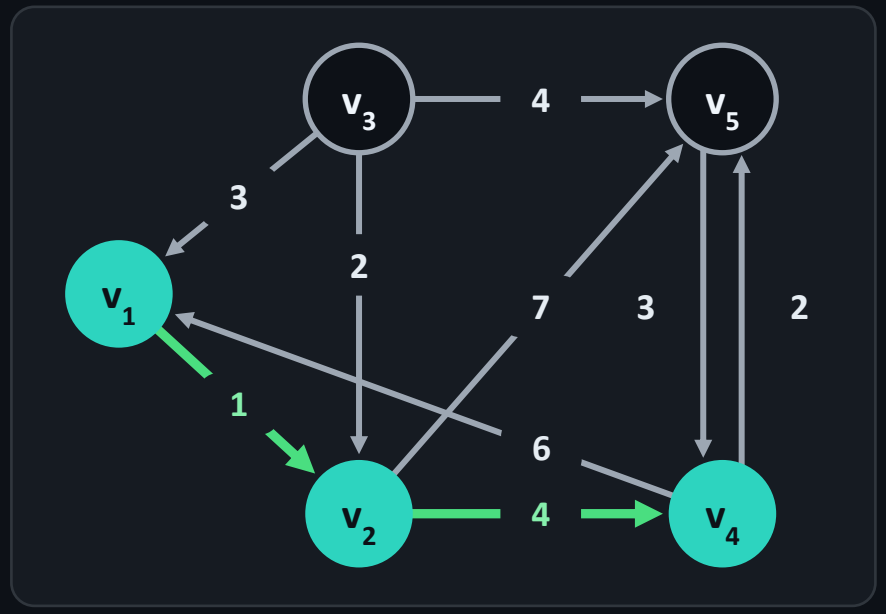
Floyd: qué cambia en cada paso k

Paso	Celdas de D que mejoran (antes → después)	Q nuevo (mismo orden)
k = 1	[4,2]: ∞ → 7	1
k = 2	[1,4]: ∞ → 5 · [1,5]: ∞ → 8 · [3,4]: ∞ → 6	2 · 2 · 2
k = 3	Ninguna	—
k = 4	[1,5]: 8 → 7 · [2,1]: ∞ → 10 · [2,5]: 7 → 6 · [5,1]: ∞ → 9 · [5,2]: ∞ → 10	4 · 4 · 4 · 4 · 1
k = 5	Ninguna	—

Regla: si `D[i][k] + D[k][j] < D[i][j]`, la celda mejora y `Q[i][j] = Q[k][j]`.

Con k = 3 no cambia nada: a v3 no llega ningún arco, así que pasar por v3 no acorta nada.

Floyd: resultado y cómo leer el camino



D5: distancias mínimas

	v1	v2	v3	v4	v5
v1	0	1	∞	**5**	7
v2	10	0	∞	4	6
v3	3	2	0	6	4
v4	6	7	∞	0	2
v5	9	10	∞	3	0

Q5: predecesores

	v1	v2	v3	v4	v5
v1	0	**1**	0	**2**	4
v2	4	0	0	2	4
v3	3	3	0	2	3
v4	4	1	0	0	4
v5	4	1	0	5	0

Cómo leer Q: `Q[i][j]` es el vértice anterior a vj en el camino de vi a vj; retroceda hasta llegar a vi.

v1 → v4: `Q[1][4] = 2` y `Q[1][2] = 1`, así que el camino es **v1 → v2 → v4**, con costo `D5[1][4] = 5`.

Warshall: ¿existe un camino?

Cierre transitivo

Calcula la matriz de camino **P** de un grafo de n vértices a partir de su matriz de adyacencia **A**.

Matrices P0 … Pn

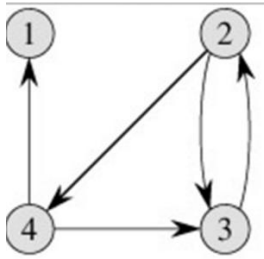
Secuencia de matrices n×n. En el paso k se permite pasar por los vértices v1 a vk.

Parecido a Floyd

Misma estructura de pasos, pero con 1 (hay camino) y 0 (no hay) en lugar de distancias.

Regla del paso k: `P[i][j] = P[i][j] OR (P[i][k] AND P[k][j])`

Warshall: ejemplo con cuatro vértices



$$T^{(0)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

$$T^{(3)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \quad T^{(4)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

T(0): arcos directos (1 en la diagonal) · T(4): cierre transitivo

Warshall: el mismo grafo de Floyd

P0: arcos y diagonal

	v1	v2	v3	v4	v5
v1	1	1	0	0	0
v2	0	1	0	1	1
v3	1	1	1	0	1
v4	1	0	0	1	1
v5	0	0	0	1	1

P5: cierre transitivo

	v1	v2	v3	v4	v5
v1	1	1	0	**1**	**1**
v2	**1**	1	0	1	1
v3	1	1	1	**1**	1
v4	1	**1**	0	1	1
v5	**1**	**1**	0	1	1

Celdas que pasan de 0 a 1 (ámbar en P5)

Paso	Celdas nuevas
k = 1	[4,2]
k = 2	[1,4] · [1,5] · [3,4]
k = 3	Ninguna
k = 4	[2,1] · [5,1] · [5,2]
k = 5	Ninguna

Con D5 de Floyd: `P5[i][j] = 1` si y solo si `D5[i][j] < ∞`. Warshall dice **si** hay camino; Floyd, **cuánto** cuesta.

// árbol de expansión mínimo

¿Qué es un árbol?

Un árbol es un subconjunto del grafo que está **conexo** y no tiene **ciclos**:

n – 1 aristas

Con n vértices, un árbol tiene exactamente n – 1 aristas.

Camino único

Entre dos vértices cualesquiera de un árbol existe un único camino.

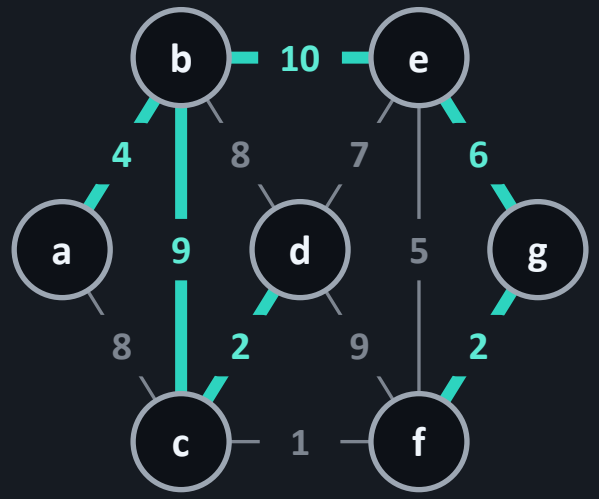
Una arista más

Agregar una arista más a un árbol produce un ciclo.

Cubre el grafo

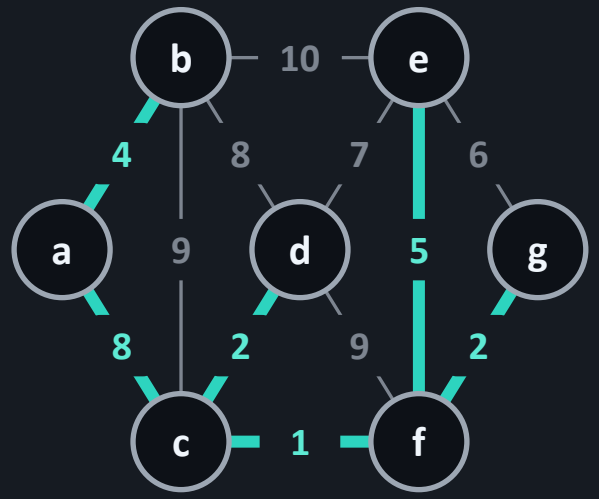
Si todos los vértices están en el árbol, el grafo es conexo.

Árbol de expansión mínimo



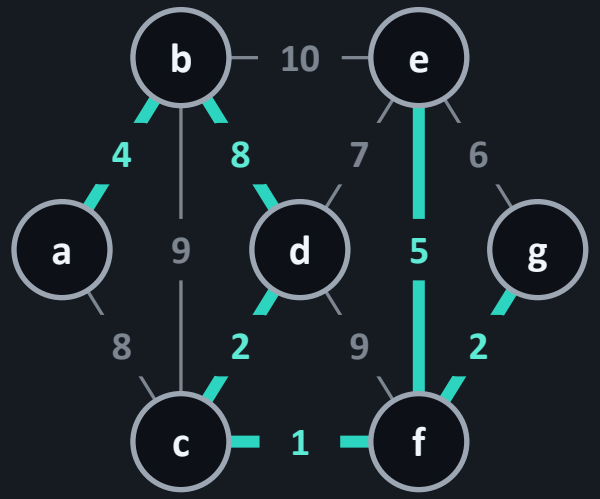
Costo = 33

No es el mínimo.



Costo = 22

Es un **árbol mínimo**.

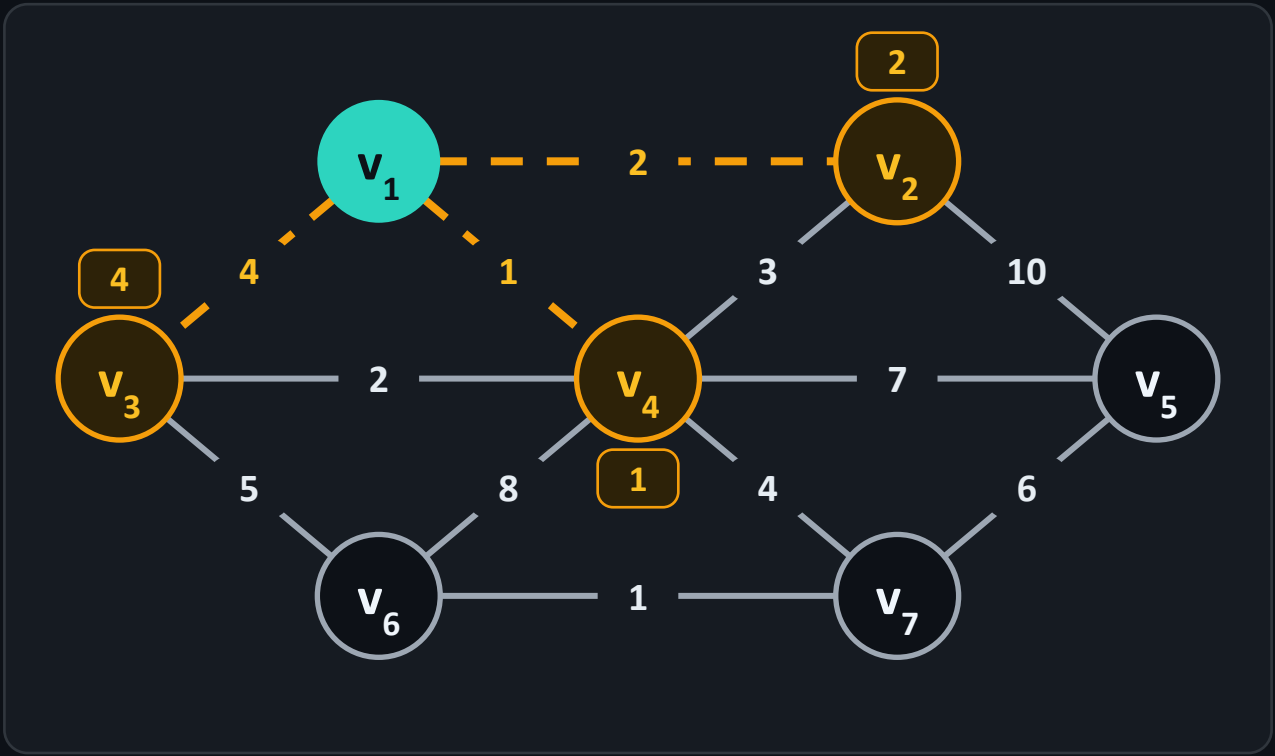


Costo = 22

Otro **árbol mínimo**.

El mismo grafo puede tener varios árboles de expansión, pero **no todos son el mínimo**.

Prim: se parte de un vértice



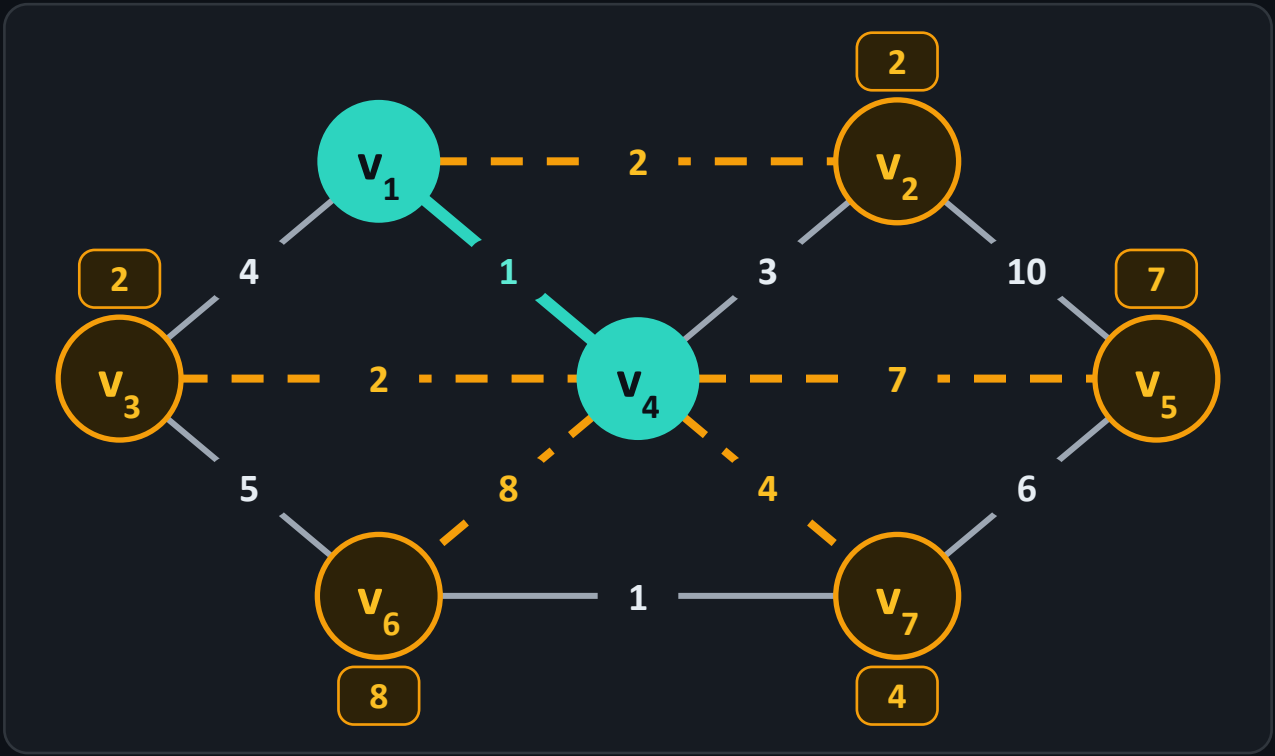
dv: peso mínimo conocido · pv: vértice previo

v	Conocido	dv	pv
v1	sí	0	—
v2	no	2	v1
v3	no	4	v1
v4	no	1	v1
v5	no	∞	—
v6	no	∞	—
v7	no	∞	—

En el árbol Candidato

Idea de Prim: el árbol crece por etapas; en cada una se agrega la arista (u, v) más barata con u en el árbol y v fuera.

Prim: se agrega v4 (dv = 1)



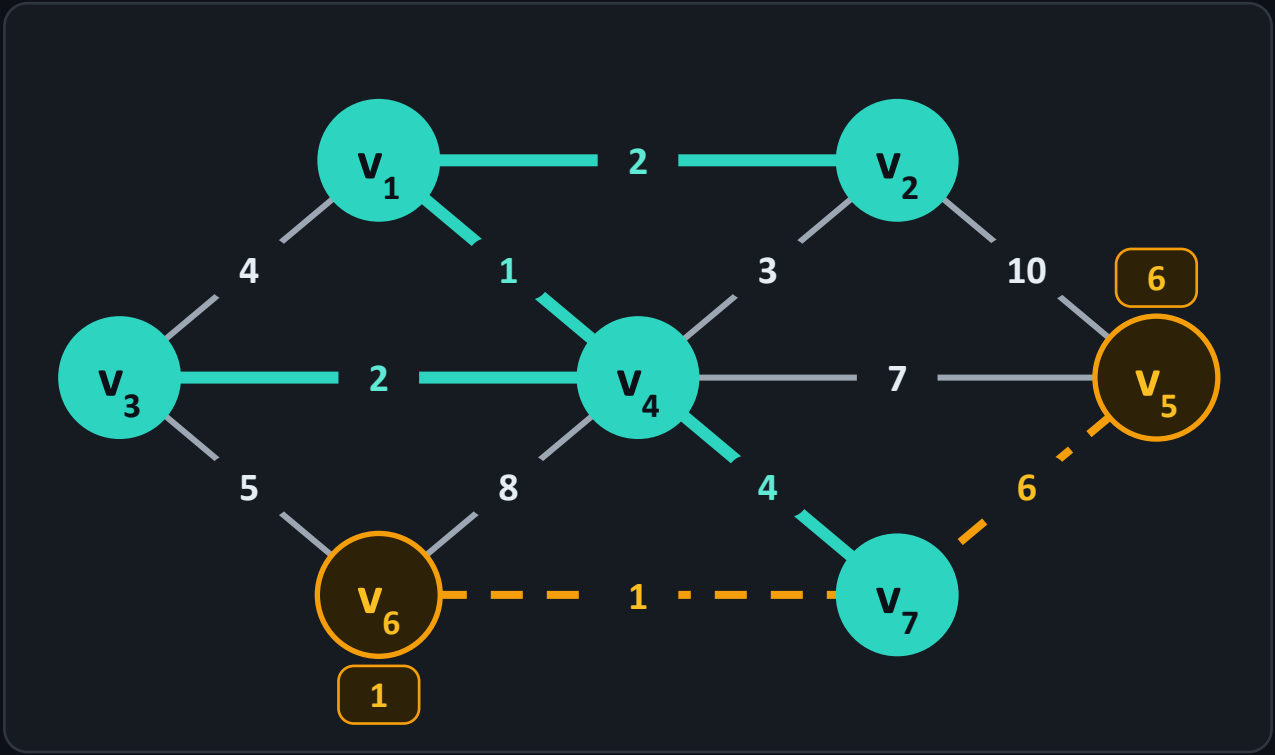
dv: peso mínimo conocido · pv: vértice previo

v	Conocido	dv	pv
v1	sí	0	—
v2	no	2	v1
v3	no	2	v4
v4	sí	1	v1
v5	no	7	v4
v6	no	8	v4
v7	no	4	v4

En el árbol Candidato

Se agrega v4 (el menor dv). Mejora v3 (4 → 2) y descubre v5 = 7, v6 = 8 y v7 = 4; v2 se queda en 2.

Prim: se agregan v2, v3 y v7



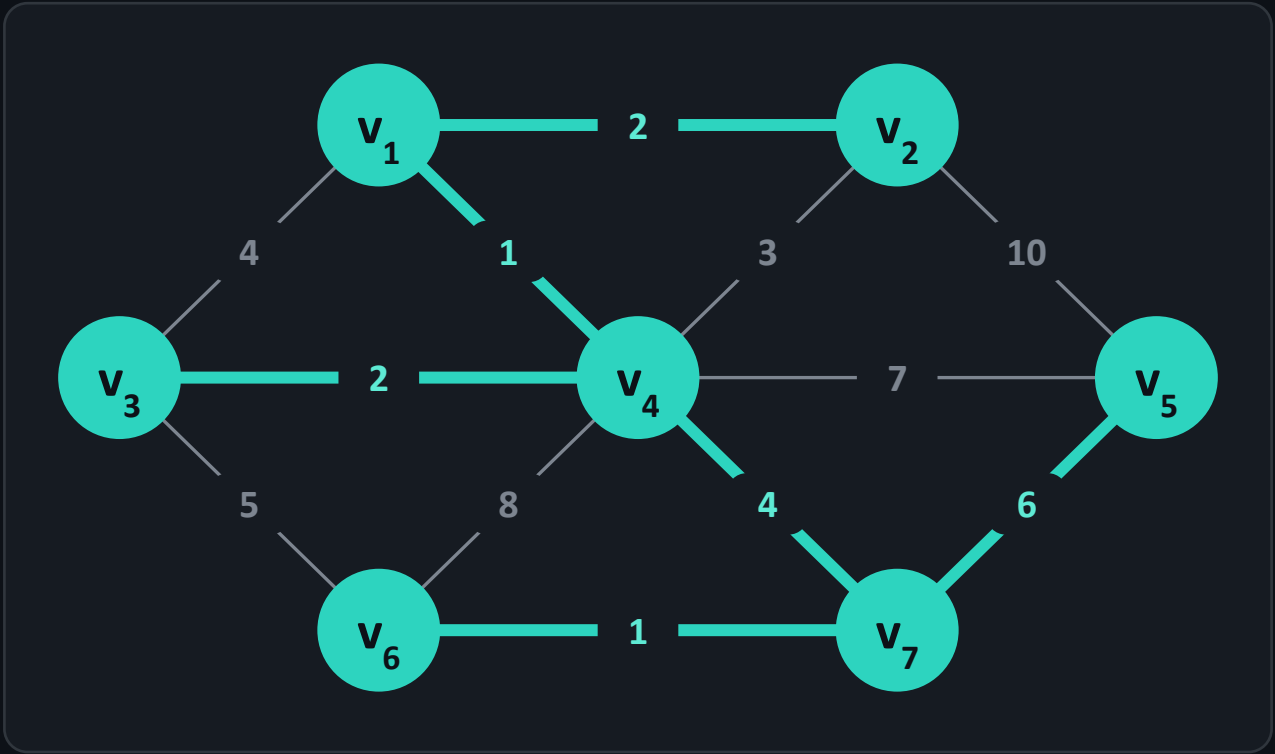
dv: peso mínimo conocido · pv: vértice previo

v	Conocido	dv	pv
v1	sí	0	—
v2	sí	2	v1
v3	sí	2	v4
v4	sí	1	v1
v5	no	6	v7
v6	no	1	v7
v7	sí	4	v4

En el árbol Candidato

v7 mejora a v6 (5 → 1) y a v5 (7 → 6). Solo faltan v6 y v5.

Prim: árbol de expansión mínimo



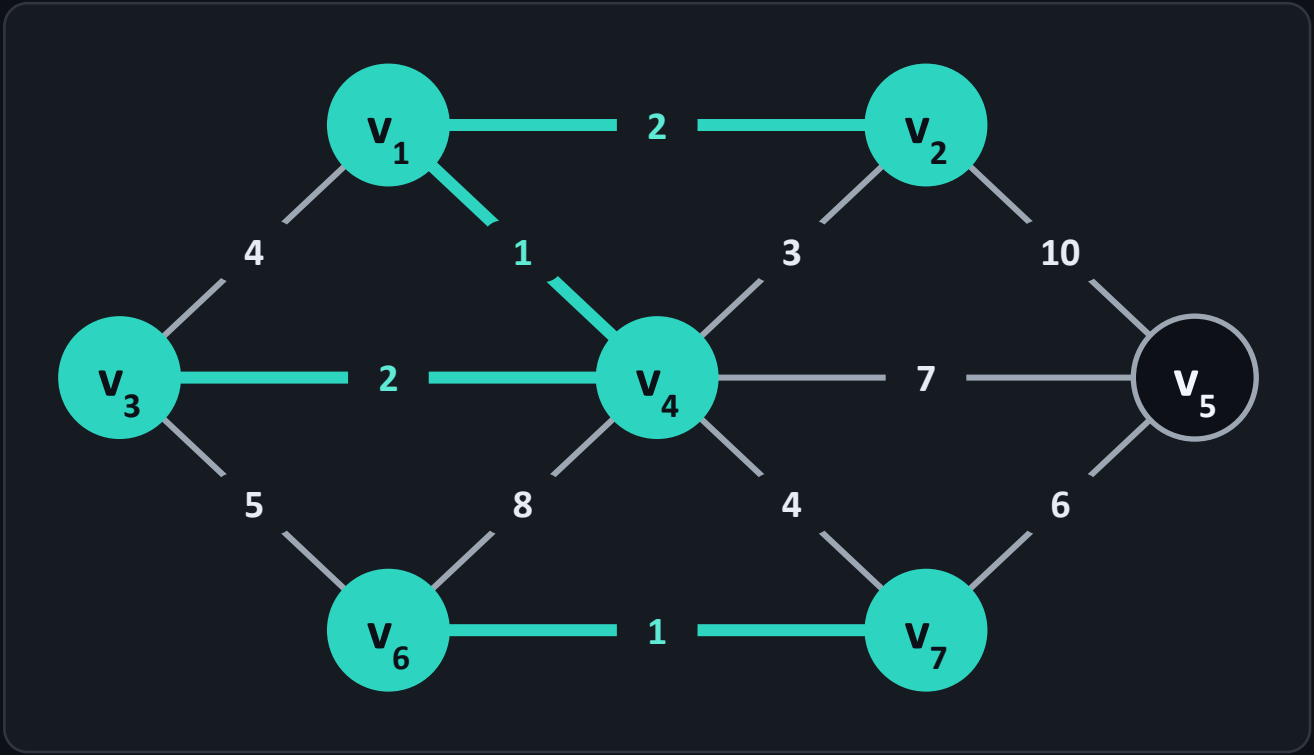
En el árbol No elegida

dv: peso mínimo conocido · pv: vértice previo

v	Conocido	dv	pv
v1	sí	0	—
v2	sí	2	v1
v3	sí	2	v4
v4	sí	1	v1
v5	sí	6	v7
v6	sí	1	v7
v7	sí	4	v4

Costo total: 1 + 2 + 2 + 4 + 1 + 6 = **16**, con 6 aristas (n − 1).

Kruskal: primeras aristas aceptadas

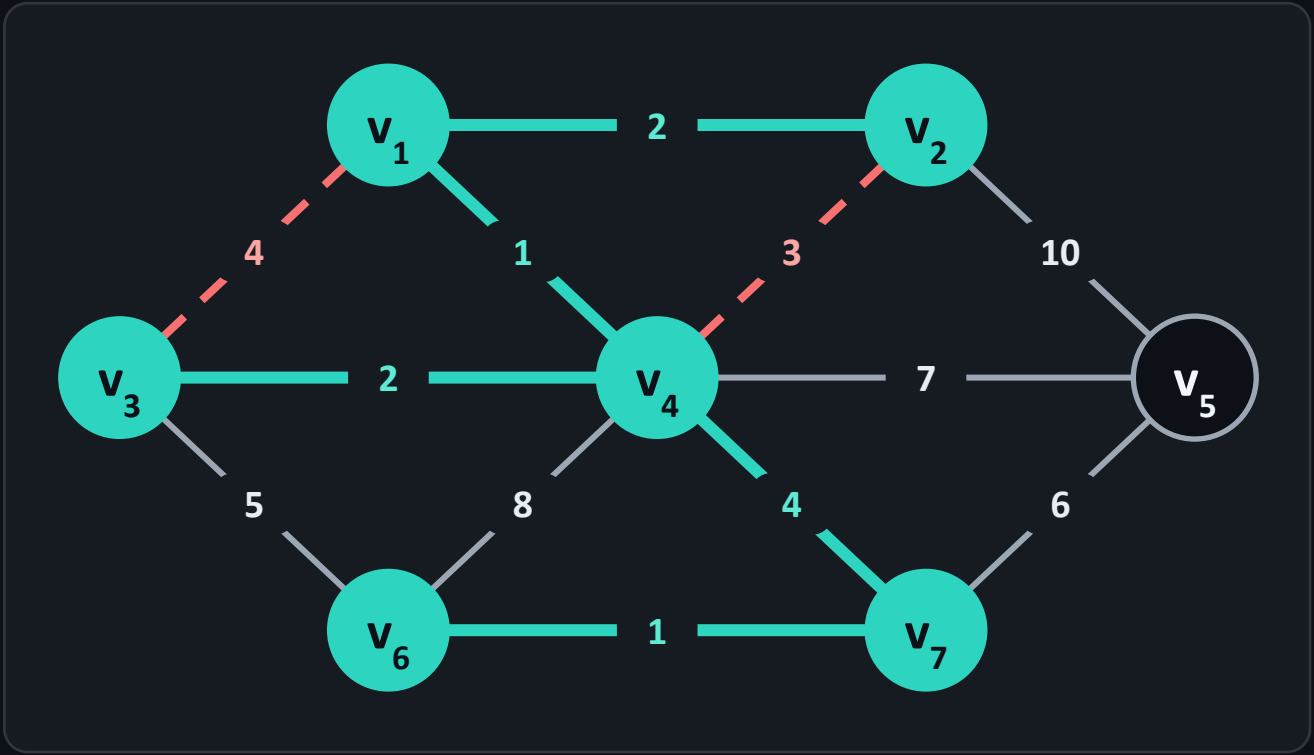


Aceptada Rechazada

Arista	Peso	Acción
(v1, v4)	1	Aceptada
(v6, v7)	1	Aceptada
(v1, v2)	2	Aceptada
(v3, v4)	2	Aceptada
(v2, v4)	3	—
(v1, v3)	4	—
(v4, v7)	4	—
(v3, v6)	5	—
(v5, v7)	6	—

Idea de Kruskal: examinar las aristas de menor a mayor peso y aceptar la que no cause un ciclo.

Kruskal: se rechazan las aristas que cierran ciclos

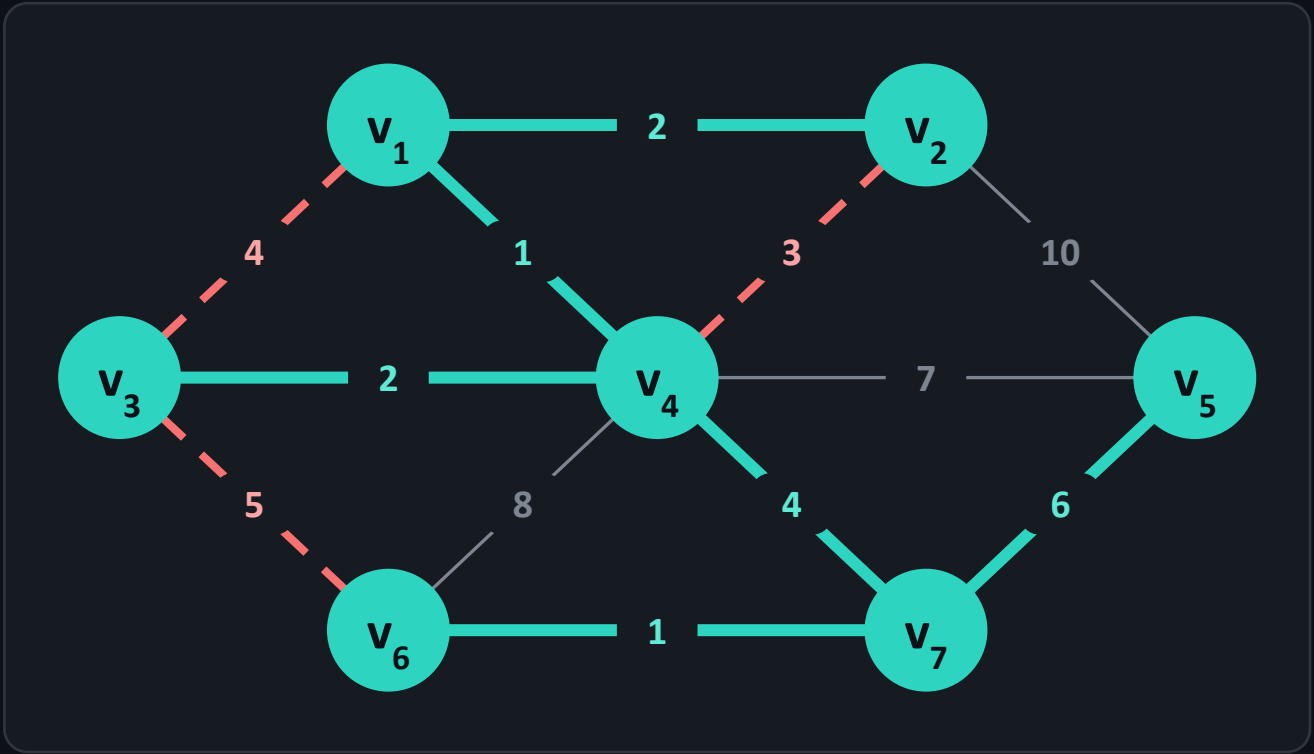


Aceptada Rechazada

Arista	Peso	Acción
(v1, v4)	1	Aceptada
(v6, v7)	1	Aceptada
(v1, v2)	2	Aceptada
(v3, v4)	2	Aceptada
(v2, v4)	3	Rechazada
(v1, v3)	4	Rechazada
(v4, v7)	4	Aceptada
(v3, v6)	5	—
(v5, v7)	6	—

(v2, v4) y (v1, v3) unirían nodos que ya están conectados: cerrarían un ciclo. (v4, v7) une dos árboles.

Kruskal: árbol de expansión mínimo



Aceptada | Rechazada | Sin examinar

Arista	Peso	Acción
(v1, v4)	1	Aceptada
(v6, v7)	1	Aceptada
(v1, v2)	2	Aceptada
(v3, v4)	2	Aceptada
(v2, v4)	3	Rechazada
(v1, v3)	4	Rechazada
(v4, v7)	4	Aceptada
(v3, v6)	5	Rechazada
(v5, v7)	6	Aceptada

Costo total: **16** con 6 aristas (n − 1), igual que Prim. Las aristas de peso 7, 8 y 10 no se examinan.

// cierre

Resumen

- **Grafo** `G = {V, A}`: dirigido o no, ponderado o no
- **Representación:** matriz (denso) o lista de adyacencia (disperso)
- **Recorridos:** Breadth-First con cola y Depth-First con pila
- **Caminos más cortos:** Dijkstra (voraz, un origen) y Floyd (todos los pares)
- **Warshall:** cierre transitivo, ¿existe camino entre cada par?
- **Expansión mínima:** Prim (crece un árbol) y Kruskal (une un bosque)

// práctica

Ahora usted

Resuelva en su cuaderno:

A

Matriz de adyacencia

Grafo dirigido con arcos 1→2, 2→3, 3→1 y 3→4. Escriba su matriz de adyacencia.

B

Recorrido BFS

En el grafo de los recorridos, haga BFS desde B (adyacentes en orden alfabético).

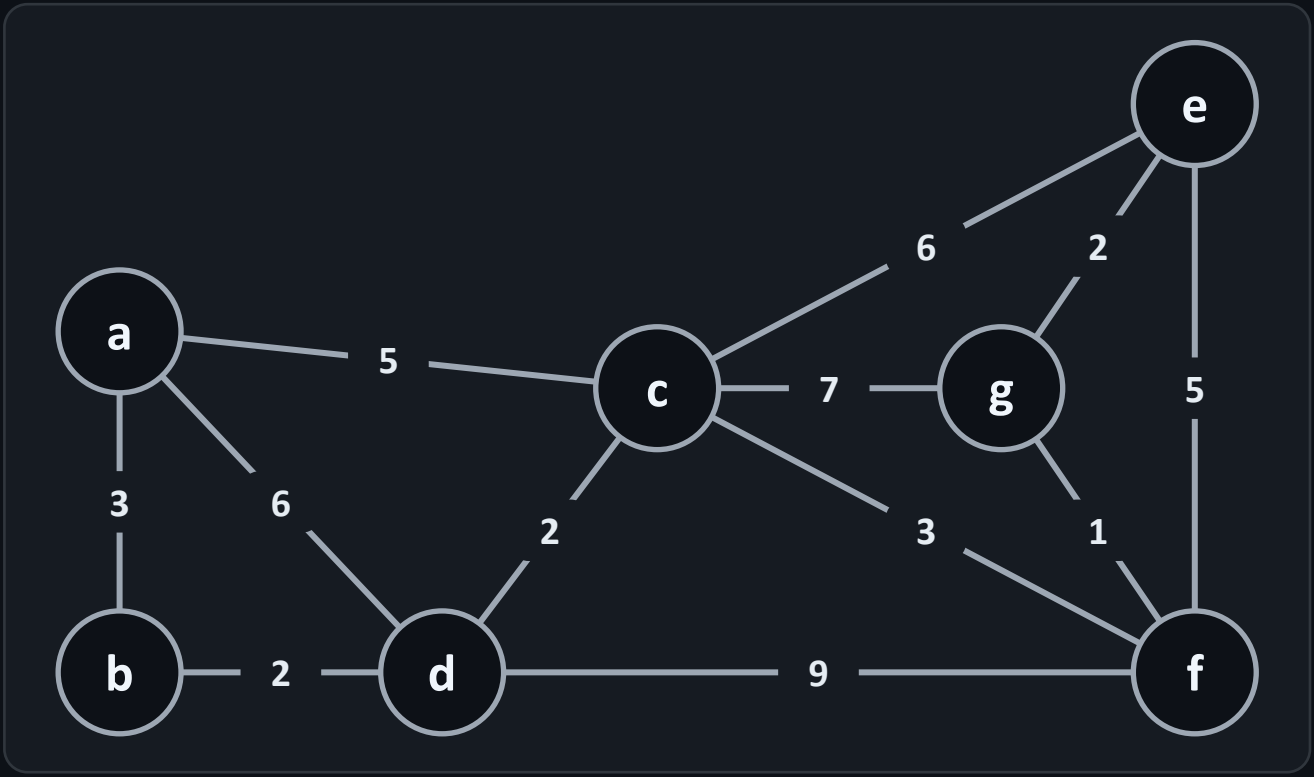
C

Árbol de expansión

Un grafo conexo tiene 8 vértices. ¿Cuántas aristas tiene su árbol de expansión mínimo?

// práctica

Ahora usted: Dijkstra



- Calcule la distancia mínima de **a** a cada vértice.
- Escriba el camino más corto de **a** a **g**.
- Lleve una tabla con vértice, temporal, final y previo.

> El grafo es no dirigido: cada arista se puede recorrer en los dos sentidos.

> **Pista:** en cada paso fije el vértice pendiente con el menor valor temporal.